did:wba方法规范

- 文档编号：ANP-03
- 标题：did:wba方法规范
- 状态：已发布
- 版本：1.2
- 语言：中文
- 适用范围：本规范适用于 ANP 中基于 Web 的去中心化身份、跨平台身份认证与智能体通信场景。

> ANP 1.2 发布说明：通用 DID 身份认证由 ANP-02 定义；DID 方法与命名规则仍由各自规范拥有。英文镜像为 did:wba Method Specification。

摘要

wba DID方法是一种基于Web的去中心化标识符（DID）规范，旨在满足跨平台身份认证和智能体通信的需求。此方法在did:web基础上进行扩展和优化，命名为did:wba，保留其兼容性并增强针对智能体场景的适配性。

在本规范中，路径型 did:wba 的默认方案会在 DID 路径中携带绑定公钥指纹，用于增强 DID 与用户自持私钥之间的绑定关系。当前版本的**默认 profile**为：

- `e1_`：绑定 Ed25519 公钥，推荐用于新部署，并可直接集成 W3C 标准的 Data Integrity EdDSA proof。

为了兼容钱包生态和现有 secp256k1 实现，本规范在附录 A 中额外定义了一个**非默认兼容扩展 profile**：

- `k1_`：绑定 secp256k1 公钥，主要用于兼容钱包生态和现有 Web3 密钥体系。

通用 HTTP/JSON 身份认证现由 ANP-02 定义；本规范提供 WBA 方法规则。原生 Web 的方法绑定、命名和消息组合见附录 B。

1. 引言

1.1 前言

wba DID方法规范符合去中心化标识符V1.0[DID-CORE]中指定的要求。

本规范在 did:web 的基础上定义 WBA 文档、绑定和生命周期规则，并通过 ANP-02 接入通用身份认证。

考虑到 did:web 方法主要面向原生 Web DID 场景，未来演进可能不完全适合智能体通信场景。另外，本规范对其进行了面向智能体通信的定向修改，和原作者就这些修改达成共识也是一个长期过程，所以我们决定使用一个新的方法名。

未来不排除将did:wba规范合并到did:web规范中的可能，我们会去推动这个目标的实现。

did:wba方法参考的did:web方法规范地址为https://w3c-ccg.github.io/did-method-web，版本日期为2024年7月31日。为了方便管理，我们备份了一份did:wba当前使用的did:web方法规范文档：did:web方法规范。

1.2 设计原则

设计did:wba方法时，我们的核心原则是即可以充分利用现有的成熟技术和完善的 Web 基础设施，又可以实现去中心化。使用did:wba，可以实现类似email特点，各个平台以中心化的方式实现自己的账户体系，同时，各个平台之间可以互联互通。

对于路径型 DID，本规范将“在 DID 路径中携带绑定公钥指纹”定义为默认方案。这样做的主要目的是让用户能够真正掌握自己的私钥，并让 DID 与用户实际控制的公钥形成稳定、可独立校验的绑定关系。即使平台维护 DID 文档的托管服务，用户或验证者仍可以根据 DID 本身验证其是否对应预期公钥，从而降低平台静默替换身份公钥的风险。

为了在标准互操作性和生态兼容性之间取得平衡，当前版本采用“**主规范默认 e1，*兼容扩展支持 k1***”的结构：

- `e1_`：绑定 Ed25519 公钥，推荐用于新部署。该 profile 可以直接与 W3C Data Integrity EdDSA 标准集成，适合作为 did:wba 的长期标准化主线。
- `k1_`：绑定 secp256k1 公钥，不作为主规范默认方案，而是在附录 A 中作为兼容钱包生态、现有 Web3 密钥体系以及依赖 secp256k1 的实现的扩展能力。

公钥指纹路径方案会带来一个自然结果：当绑定密钥发生变化时，路径型 DID 也会发生变化。因此，did:wba 不把“稳定的用户可读标识”直接压在 DID 字符串上，而是通过名称服务方案（如 WNS/Handle）来解决稳定引用问题：DID 负责“可验证的加密身份”，名称服务负责“稳定的人类可读名称”。

对于需要在绑定密钥变化后保持主体连续性的路径型 DID，最后一个绑定指纹 segment 之前的路径称为**稳定主体路径**。例如，`did:wba:example.com:user:alice:e1_<fingerprint>` 的稳定主体路径为 `example.com:user:alice`。稳定主体路径不是另一个 DID，也不能独立解析；一旦用于标识持续主体，就必须（MUST）永久不可修改、不可回收、不可重新分配。

具有相同稳定主体路径的两个完整 DID 仍然是两个不同的 DID。相同路径只是判断主体连续性的必要条件；验证者只有在按本规范验证 `successorDid` 文档链及相应 DID Document `proof` 后，才可以将它们视为同一个持续主体。

此外，各种类型的标识符系统都可以添加对 DID 的支持，从而在集中式、联合式和去中心化标识符系统之间架起互操作的桥梁。这意味着现有的中心化标识符系统无需彻底重构，只需在其基础上创建 DID，即可实现跨系统互操作，从而大大降低了技术实施的难度。

2. WBA DID 方法规范

2.1 方法名称

用于标识此DID方法的名称字符串是:wba。使用此方法的DID必须以以下前缀开头:did:wba。根据DID规范,此字符串必须是小写的。DID的其余部分(前缀之后)在下面指定。

2.2 方法特定标识符

方法特定标识符是由 TLS 保护的完全限定域名（FQDN），可以选择包含 DID 文档的路径。描述有效域名语法的正式规则在（RFC1035）、（RFC1123）和（RFC2181）中有说明。

方法特定标识符必须依据现代 TLS 服务身份校验规则与服务端证书匹配。域名匹配必须（MUST）以证书 `subjectAltName` 扩展中的 DNS 标识（`dNSName`）为准；实现不得（MUST NOT）依赖 Common Name（CN）作为服务身份匹配依据。方法特定标识符不得包含 IP 地址。可以包含端口号，但主机和端口之间的冒号必须进行百分号编码，以防止与路径发生冲突。目录和子目录可以选择性地包含，使用冒号而不是斜杠作为分隔符。

did:wba 支持两种形式：

1. 裸域名 DID：用于标识整个域名主体，也可以用于域级服务身份，例如跨域服务到服务 HTTP 身份认证；
2. 路径型 DID：用于标识域名下的具体用户、智能体或子身份。

对于**新创建的路径型 did:wba**，本规范将“路径最后一个 segment 携带绑定公钥指纹”定义为默认方案。位于指纹段之前的 path segment 由实现者自行定义，例如 `user:alice`、`agents:billing` 等。

ABNF 定义如下：

```
base64url-char = ALPHA / DIGIT / "-" / "_"
path-segment   = 1*(ALPHA / DIGIT / "-" / "_" / ".")
e1-fingerprint = "e1_" 43base64url-char

wba-root-did = "did:wba:" domain-name
wba-path-did = "did:wba:" domain-name 1*(":" path-segment) ":" e1-fingerprint
wba-did      = wba-root-did / wba-path-did
```

> 说明：
>
> 1. 主规范默认路径 profile 仅定义 `e1_`。
> 2. 如果实现需要兼容 secp256k1 路径绑定，请参见附录 A 的 `k1_` 兼容扩展。

裸域名 DID 的使用方式

`did:wba:{domain}` 的使用方式与 `did:web:{domain}` 类似，主要规则如下：

1. 解析方式与 did:web 的裸域名入口一致：
   `did:wba:example.com` 对应 `https://example.com/.well-known/did.json`
2. 裸域名 DID 主要用于表达“整个域名主体”或“域级服务身份”，而不是某个具体用户或子身份；
3. 在 ANP 的跨域服务到服务调用中，若某个 `ANPMessageService` 需要声明自己用于外层 HTTP 身份认证的 DID，则 **SHOULD** 优先使用裸域名 DID；
4. 裸域名 DID 不携带 `e1_` 路径绑定指纹，因此不适用本规范对 `e1_` 路径型 DID 的路径绑定校验规则；
5. 请求认证遵循 ANP-02 的通用 HTTP 认证与 WBA 绑定；裸域形式不免除其摘要、签名覆盖、认证用途、时间和重放要求。原生 Web 使用 ANP-02 的 Web 绑定。

关于原生 `did:web` 如何在 ANP 中以相同方式参与上述流程，详见附录 B。

2.2.1 默认路径方案：`e1_` 绑定公钥指纹

对于新创建的路径型 did:wba，最后一个 path segment 必须（MUST）是 `e1_` 绑定公钥指纹段。`e1_` 表示绑定密钥为 Ed25519 公钥，并且该 DID 采用主规范默认 profile。

推荐结构如下：

```
did:wba:{domain}:{namespace...}:{e1-fingerprint}
```

示例：

```
did:wba:example.com
did:wba:example.com:user:alice:e1_<fingerprint>
did:wba:example.com%3A3000:user:alice:e1_<fingerprint>
```

为兼容已有部署，解析器可以（MAY）支持不带指纹段的历史路径型 DID 的解析；但新创建的路径型 DID 应当（SHOULD）采用本规范定义的默认方案。

2.2.2 `e1_` 指纹生成方法（推荐）

`e1_` 指纹用于将路径型 DID 与 Ed25519 绑定公钥关联起来。其生成方法如下：

1. 选择 DID 绑定密钥。绑定密钥必须（MUST）同时满足以下条件：
    - 是一个 Ed25519 公钥；
    - 在 DID 文档中以 `Multikey` / `publicKeyMultibase` 表示；
    - 被 DID 文档的 `authentication` 关系授权。
2. 将该 Ed25519 `publicKeyMultibase` 转换为等价公钥 JWK。等价 JWK 只保留 RFC 7638 要求的必要字段：

```
{
  "crv": "Ed25519",
  "kty": "OKP",
  "x": "..."
}
```

其中：

- `publicKeyMultibase` 必须是 Multibase base58-btc 编码的 Ed25519 `Multikey`；
- 解码后得到 Ed25519 公钥原始 32 字节；
- `x` 为该 32 字节公钥的 base64url（无 padding）表示。

3. 按 RFC 7638 的规则生成 JWK Thumbprint 输入：
    - 仅保留必要字段；
    - 字段名按字典序排序；
    - 使用无多余空白的 JSON 字符串；
    - 使用 UTF-8 编码。
4. 对第 3 步得到的 UTF-8 字节序列执行 SHA-256 哈希，得到 32 字节摘要值。
5. 对 32 字节摘要值进行 base64url 编码，并去掉尾部 `=` padding。编码结果长度固定为 43 个字符。
6. 在编码结果前面添加 `e1_` 前缀，得到最终路径段。

说明：

1. `e1_` 前缀不是哈希输出的一部分，而是 profile 前缀；
2. 推荐将不带 `e1_` 前缀的 thumbprint 值用作该验证方法的 `kid` 或 fragment，便于 DID 路径与验证方法标识保持直观对应；
3. 新部署应当（SHOULD）优先采用 `e1_` profile。

2.2.3 稳定主体路径

对于采用绑定指纹路径方案的路径型 DID，稳定主体路径由域名及最后一个绑定指纹 segment 之前的全部路径 segment 组成：

```
did:wba:example.com:user:alice:e1_<fingerprint>
        └──── 稳定主体路径：example.com:user:alice ────┘
```

稳定主体路径必须（MUST）满足以下要求：

1. 一旦分配给某一持续主体，就不得（MUST NOT）修改、回收或重新分配给其他主体；
2. 根绑定密钥变化并产生新 DID 时，新旧 DID 的稳定主体路径必须（MUST）完全一致；
3. 验证者不得（MUST NOT）仅凭稳定主体路径相同就认定两个 DID 属于同一主体；
4. 主体连续性必须（MUST）按 2.5.2 至 2.5.5 节验证旧 DID Document 的 `successorDid`、整体 `proof` 以及新 DID 的绑定指纹。

2.4 密钥材料和文档处理

由于大多数Web服务器呈现内容的方式，特定的did:wba文档很可能会以application/json的媒体类型提供服务。如果检索到一个名为did.json的文档，应该遵循以下处理规则：

1. 如果JSON文档根部存在@context，则应根据JSON-LD规则处理该文档。如果无法处理，或者文档处理失败，则应拒绝将其作为did:wba文档。
2. 如果JSON文档根部存在@context，且通过JSON-LD处理，并且包含上下文 `https://www.w3.org/ns/did/v1`，则可以按照[did-core规范的6.3.2节]进一步将其处理为DID文档。
3. 如果不存在@context，则应按照[did-core规范6.2.2节]中指定的正常JSON规则进行DID处理。
4. 对外部资源、外部 DID 或 `serviceEndpoint` 的引用必须（MUST）使用绝对 URI。
5. 对同一 DID Document 内部验证方法的引用可以（MAY）使用相对 DID URL（如 `#key-1`）；解析器在处理该类引用时，必须以文档根 DID 作为基准进行展开。

> 注意：这包括嵌入的密钥材料和其他元数据中的外部URL，这可以防止密钥混淆攻击。

2.5 DID文档说明

除 DID 核心规范外，相关规范可能会随时间演进。本章节将展示一个用于身份验证的 DID 文档的子集。为了提高系统间的兼容性，所有标注为必须的字段，所有系统必须支持；标注为可选的字段，可以选择性支持。未列出的其他标准中定义的字段，可以选择性支持。

推荐的 e1 路径型 DID 文档示例如下：

```
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/data-integrity/v2",
    "https://w3id.org/security/multikey/v1",
    "https://w3id.org/security/suites/x25519-2019/v1"
  ],
  "id": "did:wba:example.com%3A8800:user:alice:e1_<fingerprint>",
  "verificationMethod": [
    {
      "id": "did:wba:example.com%3A8800:user:alice:e1_<fingerprint>#key-1",
      "type": "Multikey",
      "controller": "did:wba:example.com%3A8800:user:alice:e1_<fingerprint>",
      "publicKeyMultibase": "z6Mk..."
```

```
    },
    {
      "id": "did:wba:example.com%3A8800:user:alice:e1_<fingerprint>#key-x25519-1",
      "type": "X25519KeyAgreementKey2019",
      "controller": "did:wba:example.com%3A8800:user:alice:e1_<fingerprint>",
      "publicKeyMultibase": "z9hFgmPVfmBZwRvFEyniQDBkz9LmV7gDEqytWyGZLmDXE"
    }
  ],
  "authentication": [
    "did:wba:example.com%3A8800:user:alice:e1_<fingerprint>#key-1"
  ],
  "assertionMethod": [
    "did:wba:example.com%3A8800:user:alice:e1_<fingerprint>#key-1"
  ],
  "keyAgreement": [
    "did:wba:example.com%3A8800:user:alice:e1_<fingerprint>#key-x25519-1"
  ],
  "service": [
    {
      "id": "did:wba:example.com%3A8800:user:alice:e1_<fingerprint>#ad",
      "type": "AgentDescription",
      "serviceEndpoint": "https://agent-network-protocol.com/agents/example/ad.json"
    },
    {
      "id": "did:wba:example.com%3A8800:user:alice:e1_<fingerprint>#handle",
      "type": "ANPHandleService",
      "serviceEndpoint": "https://example.com/.well-known/handle/alice"
    },
    {
      "id": "did:wba:example.com%3A8800:user:alice:e1_<fingerprint>#anp",
      "type": "ANPMessageService",
      "serviceEndpoint": "https://example.com/anp",
      "serviceDid": "did:wba:example.com%3A8800"
    }
  ],
  "proof": {
    "type": "DataIntegrityProof",
    "cryptosuite": "eddsa-jcs-2022",
    "created": "2025-01-01T00:00:00Z",
    "verificationMethod": "did:wba:example.com%3A8800:user:alice:e1_<fingerprint>#key-1",
    "proofPurpose": "assertionMethod",
    "proofValue": "z..."
  }
}
```

字段解释：

- **@context**：必须字段，JSON-LD 上下文定义了DID文档中使用的语义和数据模型，确保文档的可理解性和互操作性。`https://www.w3.org/ns/did/v1` 是必须的。对于采用标准 Ed25519 proof 的 e1 文档，`https://w3id.org/security/data-integrity/v2` 与 `https://w3id.org/security/multikey/v1` 也是必须的。其他根据需要添加。
- **id**：必须字段，不可以携带IP，但是可以携带端口，携带端口时，冒号需要编码为`%3A`。后面使用冒号进

行路径分割。对于新创建的路径型 DID，最后一个 path segment 必须（MUST）是 `e1_<fingerprint>`。

- **alsoKnownAs**：DID Core 定义的可选字段。在根绑定密钥变化并生成新 DID 时，新 DID Document 可以（MAY）通过该字段引用直接前驱 DID。`alsoKnownAs` 只表达反向关联声明，不单独构成主体连续性的密码学证明。
- **deactivated**：did:wba 在直接 DID Document 获取模式下定义的可选扩展字段。值为 `true` 时，表示该完整 DID 已停止用于新的认证和业务路由，但其 DID Document 仍可用于历史验证和后继查询。
- **successorDid**：did:wba 定义的可选扩展字段，值必须（MUST）是完整 DID 字符串，用于从已被替代的 DID 指向其直接后继 DID。它不得（MUST NOT）跳过中间 DID 而直接改写为更晚的后继。
- **verificationMethod**：必须字段，包含验证方法的数组，定义了用于验证DID主体的公钥信息。对于需要支持端到端加密（E2EE）通信的场景，`verificationMethod` 中**应**同时包含签名密钥和密钥协商密钥，实现密钥分离。签名密钥用于身份认证和文档断言；密钥协商密钥（如 `X25519KeyAgreementKey2019`）用于上层协议的密钥协商或机密信息接收。两类密钥各司其职，单一密钥泄露不会同时影响身份认证和通信机密性。

  对于采用默认路径方案的路径型 DID，`verificationMethod` 中必须（MUST）至少存在一个 Ed25519 `Multikey` 作为绑定密钥，其等价公钥 JWK 的 RFC 7638 thumbprint 与 DID 路径最后的 `e1_` 指纹段完全一致。
    - **子字段**:
      - **id**：验证方法的唯一标识符。
      - **type**：验证方法的类型。
      - **controller**：控制该验证方法的DID。
      - **publicKeyJwk**：公钥信息，使用JSON Web Key格式。
      - **publicKeyMultibase**：公钥信息，使用 Multibase 格式。
- **authentication**：必须字段，列出用于身份验证的验证方法，可以是字符串或对象。对于采用默认路径方案的路径型 DID，绑定密钥必须（MUST）被 `authentication` 关系授权。默认情况下，跨平台身份认证应优先使用该绑定密钥进行签名。
- **assertionMethod**：可选字段，列出用于表达断言的验证方法。对于采用 e1 profile 且使用标准 DID Document proof 的 DID，绑定密钥或用于生成该 proof 的 Ed25519 `Multikey` 必须（MUST）被 `assertionMethod` 授权。
- **keyAgreement**：可选字段，定义了用于密钥协商的公钥信息，可以用于两个DID之间的加密通信。验证方法一般使用X25519KeyAgreementKey2019等可以用于密钥交换的密钥协商算法。`keyAgreement` 可以是字符串引用（指向 `verificationMethod` 中的条目）或嵌入式对象。对于端到端加密（E2EE）场景，此字段用于向上层协议提供密钥协商材料。上层协议可以是私聊端到端加密 Profile、群组端到端加密 Profile 或其它未来定义的安全 Overlay；本规范 **不**把 `keyAgreement` 写死为某一种特定算法流程。新部署通常 **应** 包含 `X25519KeyAgreementKey2019` 或语义等价的 X25519 条目。如果 DID 文档中没有 `keyAgreement` 或没有可供上层协议使用的协商条目，则表示该智能体不支持相关 E2EE 能力。
    - **子字段**:
      - **id**：密钥协商方法的唯一标识符。
      - **type**：密钥协商方法的类型。
      - **controller**：控制该密钥协商方法的DID。
      - **publicKeyMultibase**：Multibase格式的公钥信息。

- **service**：可选字段，定义了与DID主体关联的服务列表。
    - **id**：服务的唯一标识符。
    - **type**：服务类型。目前支持以下类型：
        - `AgentDescription`：智能体描述服务，`serviceEndpoint` 指向遵循ANP-智能体描述协议规范的文档。
        - `ANPHandleService`：Handle 绑定服务，用于 WNS（WBA Name Space）双向绑定验证。`serviceEndpoint` 必须（MUST）是位于 Handle Provider 域下、可解引用的 HTTPS 绝对 URI。详见 04-ANP-基于DID-WBA的命名空间规范。
            - 当 DID 持有者愿意公开其 Handle 时，`serviceEndpoint` 应（SHOULD）直接使用该 Handle 的标准 Resolution Endpoint（如 `https://example.com/.well-known/handle/alice`）；
            - 当 DID 持有者不愿在 DID Document 中公开其 Handle 时，`serviceEndpoint` 可以（MAY）指向 DID Confirmation Endpoint（如 `https://example.com/.well-known/handle/by-did?did=...`），该端点至少返回 `did` 与 `confirmed = true`；
            - 当返回文档包含 `handle` 且与输入 Handle 完全一致时，验证者可以完成具体 Handle 的精确反向验证；
            - 当返回文档只包含确认信息时，验证者只能确认 provider 关系，不得将其单独视为“具体 Handle 已验证绑定”，尤其不能用于需要确认具体 Handle 的安全敏感场景。
        - `ANPMessageService`：ANP 即时消息统一服务入口。若 DID 主体参与 ANP 即时消息协议，`serviceEndpoint` **MAY** 指向其统一的 ANP 消息端点；私聊、群聊、密钥材料访问、附件控制等能力由该单一服务入口承载，具体方法与能力声明遵循 ANP Profile 2 及相关 Profile。若该服务需要参与跨域服务到服务调用，则服务条目 **SHOULD** 额外声明 `serviceDid`，表示该服务在外层 HTTP 请求签名中使用的 DID；对 did:wba 部署，通常应使用裸域名 DID（如 `did:wba:example.com` 或 `did:wba:example.com%3A8800`）。
    - **serviceEndpoint**：服务的端点URL。
    - **serviceDid**：可选字段。当服务参与跨域服务到服务调用时，推荐声明该字段。其值应为 DID 字符串而非 DID URL，用于告诉对端“应当使用哪个 DID 的公钥来验证此外层 HTTP 请求签名”。
- **proof**：对于默认 `e1_` profile，`proof` 是必须字段；对于其他 profile，该字段是否出现由对应 profile 规则决定。`proof` 用于表达 DID Document 的完整性证明，证明 DID Document 在生成 proof 之后未被篡改，并表明 proof 创建时签名者控制了对应私钥。proof 本身不单独替代 DID method 解析过程，也不单独替代 `id` 一致性检查。
    - 对于默认 e1 profile，本版本定义了基于 W3C 标准的 `DataIntegrityProof` + `eddsa-jcs-2022` proof profile。

> 注意：
>
> 1. 公钥信息目前支持两种格式，`publicKeyJwk` 和 `publicKeyMultibase`。详细见 https://www.w3.org/TR/did-extensions-properties/#verification-method-properties。
> 2. 验证方法类型定义见 https://www.w3.org/TR/did-extensions-properties/#verification-method-

types。对于 e1 绑定密钥，推荐使用 `Multikey`。

6. 对于需要支持端到端加密通信的场景，建议采用密钥分离设计：签名/断言密钥与密钥协商密钥分开管理。签名/断言密钥不参与密钥协商，密钥协商密钥不参与签名。具体如何使用这些材料，由上层私聊 E2EE、群组 E2EE 等 Profile 分别定义。
7. 对于采用默认路径方案的新创建路径型 DID，绑定密钥必须（MUST）满足：
    - 使用 `Multikey` / `publicKeyMultibase` 表示；
    - 被 `authentication` 关系授权；
    - 其等价公钥 JWK 的 RFC 7638 thumbprint 与 DID 路径最后一个 `e1_` 指纹段完全一致。
8. 如果实现需要支持 secp256k1 路径绑定，请参见附录 A 的 `k1_` 兼容扩展。

2.5 DID方法操作

2.5.1 创建(注册)

did:wba方法规范没有指定具体的HTTP API操作，而是将程序化注册和管理留给各个实现方根据其Web环境的要求自行定义。

创建DID需要执行以下步骤：

1. 向域名注册商申请使用域名；
2. 在DNS查询服务中存储托管服务的位置和IP地址；
3. 如果创建的是路径型 DID，先生成 Ed25519 绑定密钥，并按照 2.2.2 节计算 `e1_` 指纹段；
4. 创建DID文档JSON-LD文件，包含合适的密钥对，并将 `did.json` 文件存储在 `.well-known` URL 下以代表整个域名，或者如果在该域名下需要解析多个DID，则存储在指定路径下。

例如，对于域名 `example.com`， `did.json` 将在以下 URL 下可用：

```
示例：创建DID
did:wba:example.com
 -> https://example.com/.well-known/did.json
```

裸域名 DID 的创建和托管方式与 did:web 的裸域名入口一致：域名所有者只需在 `/.well-known/did.json` 提供对应 DID 文档，即可把该 DID 用作域名主体身份或域级服务身份。

如果指定了可选路径而不是裸域名，且采用默认路径方案，则 `did.json` 将在带有 `e1_` 指纹段的路径下可用：

```
示例5：使用默认路径方案创建路径型 DID
did:wba:example.com:user:alice:e1_<fingerprint>
 -> https://example.com/user/alice/e1_<fingerprint>/did.json
```

如果在域名上指定了可选端口，则必须对主机和端口之间的冒号进行百分比编码，以防止与路径发生冲突。

```
示例6：使用可选路径和端口创建DID
did:wba:example.com%3A3000:user:alice:e1_<fingerprint>
 -> https://example.com:3000/user/alice/e1_<fingerprint>/did.json
```

> 说明：
> 如果实现需要使用 secp256k1 绑定路径 DID，请参见附录 A 的 `k1_` 兼容扩展。

2.5.2 读取(解析)

必须执行以下步骤来从 did:wba DID 解析 DID 文档：

- 将方法特定标识符中的 `:` 替换为 `/` 以获得完全限定的域名和可选路径。
- 如果域名包含端口，则对冒号进行百分比解码。
- 通过在预期的DID文档位置前加上 `https://` 生成HTTPS URL。
- 如果URL中未指定路径，则附加 `/.well-known`。
- 附加 `/did.json` 以完成URL。
- 使用能够成功协商安全HTTPS连接的代理执行对URL的HTTP GET请求，该代理强制执行 2.6节安全和隐私注意事项 描述的安全要求。
- 验证解析的DID文档的ID是否与正在解析的 did:wba DID 匹配。
- 对于主规范定义的、未设置 `deactivated = true` 的 `e1_` 路径型 DID，必须（MUST）按如下严格绑定关系验证：
    - DID Document 顶层 `proof` 必须存在；
    - `proof` 必须通过 `DataIntegrityProof` + `eddsa-jcs-2022` 校验；
    - `proof.verificationMethod` 指向的验证方法必须（MUST）是 Ed25519 `Multikey`（或语义等价的 Ed25519 验证方法表示）；
    - 以 `proof.verificationMethod` 对应的 Ed25519 公钥计算 RFC 7638 thumbprint，结果必须（MUST）与 DID 路径最后的 `e1_` 指纹段完全一致。
- 对于设置了 `deactivated = true` 的 `e1_` DID：
    - 验证者仍必须（MUST）在文档中找到与该 DID 路径 `e1_` 指纹匹配的原绑定密钥；
    - 若文档同时包含 `successorDid`，验证者必须（MUST）验证新旧 DID 的稳定主体路径相同；
    - 若整体 `proof` 由原绑定密钥签署，则该迁移可以被视为已验证；
    - 若整体 `proof` 由预授权恢复密钥签署，只有在验证者已持有停用前的可信 DID Document，并能确认该恢复密钥已在该可信文档的 `assertionMethod` 中被预先授权时，才可以将该迁移视为恢复验证通过；
    - 缺少顶层 `proof` 时，standalone hop 验证器可以（MAY）读取 `successorDid` 作为 `unverified` 迁移提示。resolver 只有通过已认证的同源 HTTPS 获取每份文档、验证每个直接 hop，并最终到达 binding proof 有效的 active `e1_` DID 后，才可以（MAY）把无签名 hop 报告为 `provider_asserted`。孤立的 `successorDid`，或缺少上述已认证完整链上下文的文档，仍为 `unverified`。
- 在HTTP GET请求期间执行DNS解析时，客户端应使用[RFC8484]以防止跟踪正在解析的身份。
- 对活动 `e1_` DID，上述 proof 校验不受本地策略开关影响，而是解析成功的必要条件。
- 对其他 profile，如果本地策略启用了 DID Document proof 校验，且文档包含 `proof`，则应按对应 profile 规则进行校验。

对于裸域名 DID，方法解析使用 `/.well-known/did.json` 并检查 `id` 一致性；不适用 `e1_` 路径绑定校验。请求认证另按 ANP-02 WBA 绑定及通用认证执行；本段不定义简化的验签流程。

> 说明：
> 如果实现同时支持附录 A 的 `k1_` 兼容扩展，则对 `k1_` DID 的解析和绑定验证应按附录 A 执行。

2.5.3 更新

要更新 DID 文档，需要更新DID对应的 `did.json` 文件。

对于采用默认路径方案的路径型 did:wba，只要 DID 路径最后的绑定密钥指纹不变，DID 本身将保持不变，但 DID 文档的其他内容可以更改，例如，添加新的验证密钥、撤销旧密钥或更新服务端点。

如果绑定密钥发生变化，则该路径型 DID 必须（MUST）变更为新的 DID。新旧 DID 是两个不同的 DID，但可以按以下规则建立同一持续主体的迁移关系：

1. 新 DID 必须（MUST）与旧 DID 具有完全相同的稳定主体路径；
2. 必须（MUST）创建并发布新 DID Document，新文档可以（MAY）在 `alsoKnownAs` 中引用直接前驱 DID；
3. 旧 DID Document 必须（MUST）继续可获取，并设置 `deactivated = true` 与 `successorDid = <new DID>`；
4. `successorDid` 必须（MUST）只指向直接下一代 DID；后续再次轮换时，不得改写更早 DID 的 `successorDid`；
5. 旧 DID Document 的整体 `proof` 应覆盖 `deactivated`、`successorDid` 及文档中的其他属性；新 DID Document 由新绑定密钥生成整体 `proof`；
6. 上层名称服务（如 WNS/Handle）应同步更新到新 DID，但名称服务映射本身不替代上述迁移证明。

程序化管理接口不由本规范定义。为避免并发轮换产生多个后继 DID，实现应当（SHOULD）以当前完整 DID（例如 `expected_current_did`）执行原子比较并交换，而不是仅依据稳定主体路径提交更新。

> 注意：
>
> 1. 使用诸如 git 之类的版本控制系统和诸如 GitHub Actions 之类的持续集成系统来管理 DID 文档的更新，可以为身份验证和审计历史提供支持。
> 2. HTTP API 更新过程没有指定具体的 HTTP API，而是将程序化注册和管理留给各个实现方根据其需求自行定义。

2.5.4 停用（撤销）

对于彻底停用且不存在后继 DID 的身份，可以移除 `did.json` 文件，或者由于其他原因使其不再公开可用。

对于因绑定密钥轮换而被新 DID 替代的路径型 did:wba，不得（MUST NOT）移除旧 DID Document。旧文档必须（MUST）继续可获取，并设置 `deactivated = true` 与 `successorDid`，以支持历史签名验证和后继查询。这里的 `deactivated` 只表示该完整 DID 不再用于新的认证和路由，不表示其持续主体已经消失。

2.5.5 DID Document proof

did:wba DID Document 的顶层 `proof` 字段是否出现，取决于所采用的 profile 和文档状态。对于使用默认 `e1_` profile 的活动文档，`proof` 是必须字段。设置了 `successorDid` 的已停用 `e1_` transition 文档按下文定义的 binding、recovery、已认证 Provider 和 unverified 分支处理，因此其顶层 `proof` 可以（MAY）缺失。对于其他 profile，DID Document 可以（MAY）包含顶层 `proof` 字段，用于提供文档完整性证明。该字段用于证明 DID Document 在生成 proof 之后未被篡改，并表明 proof 创建时签名者控制了对应私钥。proof 本身不单独替代 DID method 解析过程，也不单独替代 `id` 一致性检查。

对于默认 `e1_` profile，主规范定义的 `proof` profile 必须（MUST）符合：

- Verifiable Credential Data Integrity 1.0
- Data Integrity EdDSA Cryptosuites v1.0

`proof` 对象包含以下字段：

- `type`：必须字段。固定为 `DataIntegrityProof`
- `cryptosuite`：必须字段。固定为 `eddsa-jcs-2022`
- `created`：必须字段。proof 创建时间，采用 XML Schema datetime 格式
- `verificationMethod`：必须字段。完整 DID URL，指向 DID Document 中用于生成 proof 的 Ed25519 `Multikey`
- `proofPurpose`：必须字段。固定为 `assertionMethod`
- `proofValue`：必须字段。使用 base58-btc multibase（`z...`）编码
- `domain`：可选字段
- `challenge`：可选字段

额外约束：

1. 对活动 `e1_` 文档，`proof.verificationMethod` 必须（MUST）使用 `e1_` 绑定密钥，使 DID 路径绑定、公钥绑定与文档完整性证明统一；对已停用且包含 `successorDid` 的文档，该方法也可以是停用前可信文档中已由 `assertionMethod` 预授权的 recovery key，但不得使用停用时才加入的 key；
2. 对活动 `e1_` 文档，以及声明通过旧 binding key 建立连续性的已停用 transition，解析器必须（MUST）以 `proof.verificationMethod` 对应的 Ed25519 公钥重新计算 RFC 7638 thumbprint，并验证其与 DID 路径最后的 `e1_` 指纹段完全一致。recovery proof 则必须（MUST）使用停用前可信文档中的 key material 和 `assertionMethod` 授权进行验证；recovery key 不要求与 `e1_` binding 指纹一致；
3. `proof` 的生成与验证必须（MUST）遵循 `eddsa-jcs-2022` 的标准算法流程，不再由本规范重写算法细节。

解析活动 `e1_` did:wba DID Document 时，proof 校验不是可选增强检查，而是路径绑定语义的一部分；缺少 `proof`、`proof` 验证失败、或 `proof.verificationMethod` 与 `e1_` 绑定指纹不一致时，解析必须（MUST）失败。

对于 `e1_` DID，不存在“活动 DID Document 不含 `proof` 仍可继续解析”的宽松模式。

对于设置了 `deactivated = true` 和 `successorDid` 的旧 DID Document，顶层 `proof` 仍然是一个覆盖移除 `proof` 后完整文档的整体证明：

- 由旧 DID 绑定密钥签署时，验证结果为强密码学连续性；
- 由旧 DID 在停用前已通过 `assertionMethod` 授权的恢复密钥签署时，验证者必须（MUST）依据此前可信状态确认该预授权关系；
- 如果顶层 `proof` 存在，它必须（MUST）验证为旧 binding proof 或预授权 recovery proof。格式错误、未授权或密码学验证失败的 proof 必须（MUST）使 transition 失效，且不得（MUST NOT）降级为 provider assertion 或 unverified hint；
- 当旧私钥和预授权恢复密钥均不可用时，顶层 `proof` 可以（MAY）缺失。standalone hop 验证器必须（MUST）将该 hop 报告为 `unverified`。完整链 resolver 只有通过已认证的同源 HTTPS 获取 predecessor 和每个 successor、逐 hop 验证稳定主体路径与直接后继关系，并最终到达 binding proof

有效的 active `e1_` DID 后，才可以（MAY）报告 `provider_asserted`。失败或不完整的链不得（MUST NOT）产生 `provider_asserted`。

迁移 assurance 只有以下四种含义：

- `verified`：predecessor 原 binding key 对覆盖停用和直接后继关系的整体文档签名；
- `recovery_verified`：停用前可信文档通过 `assertionMethod` 预授权的 recovery key 对该关系签名；
- `provider_asserted`：已认证的同源 HTTPS Provider 解析完整、结构有效的迁移链并到达 proof 有效的 active DID，或身份 Provider 通过独立认证的 recovery/transition 权威通道提供同一 predecessor→successor 事实。该 assurance 不是 DID Document 属性，且不得（MUST NOT）提升为 `verified` 或 `recovery_verified`；
- `unverified`：只有孤立的 `successorDid`、`alsoKnownAs`、Handle/WNS 映射、HTTP 409 hint、standalone 无签名 hop、不完整链，或没有已认证 Provider 来源的文档。

对于非 `e1_` profile，实现可以（MAY）按对应 profile 规则或本地策略在以下两种模式中选择其一：

1. 宽松模式：若 DID Document 不含 `proof`，仍可继续解析；
2. 严格模式：若本地策略要求 proof，则 DID Document 缺少 `proof` 时必须（MUST）视为验证失败。

规范性说明：

对于采用 `e1_` profile 的 DID，本规范要求 DID Document 的顶层 `proof` 使用 W3C 标准的 Data Integrity proof 机制。其 proof 数据模型、proof configuration、document transformation、hashing、proof serialization 以及 verification 规则，分别遵循 Verifiable Credential Data Integrity 1.0 和 Data Integrity EdDSA Cryptosuites v1.0。本规范仅约束 did:wba 场景下 proof 的使用位置、字段要求与验证关系，不重复定义底层密码学算法；若出现冲突，以上游 W3C 规范为准。

2.6 安全和隐私注意事项

安全与隐私注意事项参考[did:web 方法规范2.6节]。实现者还应额外关注默认路径方案下绑定密钥变更带来的 DID 轮换，以及名称服务同步问题。

本版本不要求独立可验证日志。完整解析成功后，客户端应当（SHOULD）仅缓存 hop assurance 为 `verified` 或 `recovery_verified` 的后继边。`provider_asserted` 和 `unverified` 边不得（MUST NOT）进入该已验证边缓存。在 cache commit 步骤之前发生的验证失败，包括最终活动文档 proof 无效、循环和 hop 上限失败，不得（MUST NOT）在其中遗留失败链的前缀边。如果同一旧 DID 的已缓存验证边与新观察到的 `successorDid` 不同、迁移链出现循环、或后继 DID 的稳定主体路径发生变化，必须（MUST）拒绝该迁移并向上层报告冲突。

新部署应当（SHOULD）优先采用 `e1_` profile，以获得更好的标准 proof 互操作性。如果实现需要兼容钱包生态和现有 secp256k1 实现，请参见附录 A 的 `k1_` 兼容扩展。

3. 基于did:wba方法和HTTP协议的跨平台身份认证

本章通用认证见 ANP-02 第 3 章。WBA 方法规则仍由第 2 章定义；请求格式、验证流程和实现策略保持发布前认证设计不变。

3.1 WBA 请求认证的方法约束

通用请求格式与验签流程由 ANP-02 定义。本节提供 WBA 方法要求，不将其推广到其他 DID 方法。

3.1.1 认证密钥选择

对于采用 E1 profile 的 `did:wba`，默认情况下，客户端应当（SHOULD）使用 DID 路径最后 `e1_` 指纹段对应的**绑定密钥**进行签名。服务端如果允许其他 `authentication` 验证方法，属于本地授权策略，不改变 DID 的绑定语义。

> 说明：
> 如果实现同时支持附录 A 的 `k1_` 兼容扩展，则对 `k1_` DID 的认证签名可按附录 A 执行。

3.1.2 方法状态与身份绑定

读取 DID 文档：根据 DID 解析 DID 文档。若文档设置 `deactivated = true`，服务端不得（MUST NOT）继续使用该 DID 完成新的身份认证；当文档包含 `successorDid` 时，应按 本章的 409 响应返回 DID 已被替代的错误。

- 对 `did:wba` 的 `e1_` DID，必须（MUST）基于 `proof.verificationMethod` 对应的 Ed25519 公钥验证 DID 绑定关系，而不是在 `authentication` 中任意选择一个 Ed25519 key：
  - `proof` 必须存在并通过 `eddsa-jcs-2022` 校验；
  - 用该公钥重新计算 RFC 7638 thumbprint，结果必须（MUST）与 DID 路径最后的 `e1_` 指纹段完全一致。

> 说明：
> 如果实现同时支持附录 A 的 `k1_` 兼容扩展，则对 `k1_` DID 的绑定验证和认证验证应按附录 A 执行。

3.2 WBA 的 HTTP 409 DID Superseded 响应

当请求目标 DID 或用于认证的 DID 已设置 `deactivated = true`，且存在后继 DID 时，服务端可以返回 `409 Conflict`：

```
HTTP/1.1 409 Conflict
Content-Type: application/json
Cache-Control: no-store
```

```
{
  "code": "did_superseded",
  "requestedDid": "did:wba:example.com:user:alice:e1_<old-fingerprint>",
  "currentDid": "did:wba:example.com:user:alice:e1_<current-fingerprint>",
  "stableSubjectId": "example.com:user:alice"
}
```

响应中的 `currentDid` 只作为未验证提示。客户端不得（MUST NOT）直接依据该字段重试，而应获取旧 DID Document，验证 `deactivated`、`successorDid` 和整体 `proof`，再逐个解析后继 DID，直到找到当前活动 DID。客户端应使用当前 DID 重新生成签名后再发起请求，不应使用透明 `301` / `302` 跳转代替该验证流程。

4. 基于did:wba方法和json格式数据承载的跨平台身份认证流程

本章认证信息承载见 ANP-02 第 4 章，保留原有字段、序列化约定、组件映射与验证流程。

5 区分人类授权与智能体自动授权

参见 ANP-02 第 5 章。

6 隐私保护策略

参见 ANP-02 第 6 章。

对于需要稳定对外引用、但又希望底层 DID 可轮换的场景，推荐结合名称服务（如 WNS/Handle）使用：Handle 保持稳定和人类可读，底层 did:wba 可以随着绑定密钥变化而轮换。

7 安全性建议

参见 ANP-02 第 7 章。方法专属规则继续引用本规范第 2 章，本次拆分不增加安全策略要求。

WBA 方法专属安全要求：

- 对于采用默认路径方案的 `did:wba`，绑定密钥最好使用硬件隔离、HSM 或系统安全区保管。
- 对于采用绑定密钥的 `did:wba`，默认情况下，跨平台身份认证应优先使用绑定密钥进行签名。
- 新的 WBA 部署应当（SHOULD）优先采用 `e1_` profile。
- 当绑定密钥发生变化时，路径型 DID 会随之变化，因此**应该**同步更新上层名称服务映射。
- 对于活动的 `did:wba` E1 DID，解析器必须（MUST）验证 `DataIntegrityProof`。已停用 `e1_` transition 文档按 2.5.5 节的 binding、recovery、provider-asserted 和 unverified 规则处理；对于其他 profile，如果实现启用了 DID Document proof 校验，则应当（SHOULD）按对应 profile 规则验证 `proof`。
- WBA 的稳定主体路径不得（MUST NOT）被回收或重新分配；验证者不得（MUST NOT）仅通过删除最后一个绑定指纹 segment 来合并两个 DID。
- 验证者按 WBA 方法跟随 `successorDid` 时必须（MUST）限制链长、检测循环，并拒绝稳定主体路径不一致或同一旧 DID 出现多个后继的情况。

8. 用例

1. **用例 1：用户通过智能助理访问其他网站上的文件**

Alice在example.com网站上存储了一个文件，后来她希望通过智能助理访问该文件。为此，Alice首先在智能助理上创建了一个基于did:wba方法的DID，并登录到example.com，将这个DID与自己的账户关联，并授予DID访问文件的权限。完成设置后，智能助理就可以使用该DID登录example.com，经过身份验证后，example.com允许智能助理访问Alice存储的文件。这个DID也可以配置到其他网站，以便智能助理访问不同平台上的文件。

2. **用例 2：用户通过智能助理调用其他平台服务的API**

Alice希望通过智能助理调用一个名为example的第三方服务API。首先，Alice在智能助理上创建了一个基于did:wba方法的DID，并使用该DID订购了example平台的相关服务。example服务通过DID完成身份认证，确认购买者是Alice，并记录下她的DID。认证通过后，Alice便可以通过智能助理使用该DID调用example服务的API进行操作。

> 当前用例中并未列举客户端对服务端的身份认证，事实上这个流程也是可以工作的。

9. 总结

本规范在 did:web 的基础上定义 WBA 文档、绑定和生命周期规则，并通过 ANP-02 接入通用身份认证。

本版进一步将路径型 DID 的默认方案定义为“在 DID 路径中携带绑定公钥指纹”，主规范默认采用 `e1_` profile：

- `e1_`：绑定 Ed25519 公钥，推荐用于新部署，并可直接集成 W3C 标准的 Data Integrity EdDSA proof。

为了兼容钱包生态和现有 secp256k1 实现，本规范在附录 A 中额外定义了 `k1_` 兼容扩展。

当绑定密钥变化并生成新 DID 时，本规范以永久稳定主体路径作为连续性锚点，以旧 DID Document 的 `successorDid` 和整体 `proof` 证明前向迁移；新 DID Document 可以通过 `alsoKnownAs` 反向声明旧 DID。相同稳定主体路径本身不构成 DID 等价证明。

通用 HTTP/JSON 请求认证和可选 Token 的唯一规范性定义位于 ANP-02。

未来 WBA 修订可以演进方法能力和文档服务声明；通用认证与 Token 扩展通过 ANP-02 演进。

附录 A：`k1_` 兼容扩展（非默认）

参考文档附录A：did-wba-k1_兼容扩展.md

附录 B：原生`did:web` 兼容方案

参考文档附录B：与原生did-web-的兼容.md

参考文献

1. **DID-CORE**. Decentralized Identifiers (DIDs) v1.0. Manu Sporny; Amy Guy; Markus Sabadello; Drummond Reed. W3C. 19 July 2022. W3C Recommendation. Retrieved from https://www.w3.org/TR/did-core/
2. **did:web**. Retrieved from https://w3c-ccg.github.io/did-method-web/
3. **RFC 7638**. JSON Web Key (JWK) Thumbprint. M. Jones; N. Sakimura. IETF. September 2015. Internet Standards Track. Retrieved from https://www.rfc-editor.org/rfc/rfc7638
4. **RFC 8785**. JSON Canonicalization Scheme (JCS). A. Rundgren; B. Jordan; S. Erdtman. IETF. June 2020. Informational. Retrieved from https://www.rfc-editor.org/rfc/rfc8785
5. **RFC 1035**. Domain names - implementation and specification. P. Mockapetris. IETF. November 1987. Internet Standard. Retrieved from https://www.rfc-editor.org/rfc/rfc1035
6. **RFC 1123**. Requirements for Internet Hosts - Application and Support. R. Braden, Ed. IETF. October 1989. Internet Standard. Retrieved from https://www.rfc-editor.org/rfc/rfc1123
7. **RFC 2119**. Key words for use in RFCs to Indicate Requirement Levels. S. Bradner. IETF. March 1997. Best Current Practice. Retrieved from https://www.rfc-editor.org/rfc/rfc2119
8. **RFC 2181**. Clarifications to the DNS Specification. R. Elz; R. Bush. IETF. July 1997. Proposed Standard. Retrieved from https://www.rfc-editor.org/rfc/rfc2181
9. **RFC 8174**. Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. B. Leiba. IETF. May 2017. Best Current Practice. Retrieved from https://www.rfc-editor.org/rfc/rfc8174
10. **RFC 8484**. DNS Queries over HTTPS (DoH). P. Hoffman; P. McManus. IETF. October 2018. Proposed Standard. Retrieved from https://www.rfc-editor.org/rfc/rfc8484
11. **RFC 7519**. JSON Web Token (JWT). M. Jones; J. Bradley; N. Sakimura. IETF. May 2015. Internet Standards Track. Retrieved from https://www.rfc-editor.org/rfc/rfc7519
12. **RFC 9110**. HTTP Semantics. R. Fielding, Ed.; M. Nottingham, Ed.; J. Reschke, Ed. IETF. June 2022. Internet Standard. Retrieved from https://www.rfc-editor.org/rfc/rfc9110
13. **RFC 9421**. HTTP Message Signatures. A. Backman; M. Prorock; A. Sporny. IETF. February 2024. Internet Standards Track. Retrieved from https://www.rfc-editor.org/rfc/rfc9421
14. **RFC 9530**. Digest Fields. R. Polli; L. Pardue. IETF. February 2024. Internet Standards Track. Retrieved

from https://www.rfc-editor.org/rfc/rfc9530

15. **RFC 9525**. Service Identity in TLS. P. Saint-Andre; R. Bonica; J. Hodges. IETF. November 2023. Internet Standards Track. Retrieved from https://www.rfc-editor.org/rfc/rfc9525
16. **DID Use Cases**. Decentralized Identifier Use Cases. Joe Andrieu; Kim Hamilton Duffy; Ryan Grant; Adrian Gropper. W3C. 24 June 2021. W3C Note. Retrieved from https://www.w3.org/TR/did-use-cases/
17. **DID Extensions**. Decentralized Identifier Extensions. Orie Steele; Manu Sporny. W3C. 24 June 2021. W3C Note. Retrieved from https://www.w3.org/TR/did-extensions/
18. **DID Extension Properties**. Decentralized Identifier Extension Properties. Orie Steele; Manu Sporny. W3C. 24 June 2021. W3C Note. Retrieved from https://www.w3.org/TR/did-extensions-properties/
19. **DID Extension Methods**. Decentralized Identifier Extension Methods. Orie Steele; Manu Sporny. W3C. 24 June 2021. W3C Note. Retrieved from https://www.w3.org/TR/did-extensions-methods/
20. **DID Extension Resolution**. Decentralized Identifier Extension Resolution. Orie Steele; Manu Sporny. W3C. 24 June 2021. W3C Note. Retrieved from https://www.w3.org/TR/did-extensions-resolution/
21. **Controller Document**. Controller Document. Manu Sporny; Markus Sabadello. W3C. 24 June 2021. W3C Note. Retrieved from https://www.w3.org/TR/controller-document/
22. **VC-DATA-INTEGRITY**. Verifiable Credential Data Integrity 1.0. W3C. Retrieved from https://www.w3.org/TR/vc-data-integrity/
23. **VC-DI-EDDSA**. Data Integrity EdDSA Cryptosuites v1.0. W3C. Retrieved from https://www.w3.org/TR/vc-di-eddsa/
24. **VC-DI-ECDSA**. Data Integrity ECDSA Cryptosuites v1.0. W3C. Retrieved from https://www.w3.org/TR/vc-di-ecdsa/
25. **CID-1.0**. Controlled Identifiers v1.0. W3C. Retrieved from https://www.w3.org/TR/cid-1.0/

版权声明

Copyright (c) 2024 ANP Community
本文件依据 Apache License 2.0 发布，您可以自由使用和修改，但必须保留本版权声明。